CO1 – AT3

Course Code: MLA0201 **Name:** Siva Balan K

Course Name: Fundamentals of Machine Learning **Reg.no:** 192572037

Experiments

EXPERIMENT 1 - Probability Theory Using Breast Cancer Dataset

Problem Description

The objective is to calculate the prior probability of benign and malignant breast-cancer classes. Gaussian Naive Bayes is then used to calculate posterior probabilities for an unseen patient record and predict whether the tumour is benign or malignant.

The classes are:

B = Benign

M = Malignant

Dataset Description

The Breast Cancer Wisconsin Diagnostic Dataset contains **569 patient records** and **30 numerical medical features** obtained from digitized images of breast-mass cell nuclei. The target column diagnosis contains B for benign and M for malignant.

Dataset source: Breast Cancer Wisconsin Diagnostic Dataset – Kaggle

The downloaded file is:

data.csv

Keep the files together:

EXP-1

├── probability_cancer.py

└── data.csv

Algorithm

1. Load the Breast Cancer Wisconsin dataset.
2. Remove ID, empty columns and missing records.
3. Convert benign and malignant classes into 0 and 1.

4. Calculate the prior probability of each class.
5. Split the dataset into training and testing records.
6. Train a Gaussian Naive Bayes classifier.
7. Calculate posterior probabilities and predict an unseen record.

Python Code

```
from pathlib import Path

import pandas as pd
from sklearn.metrics import accuracy_score
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB



print("Probability Theory Using Breast Cancer Dataset")
print("-" * 55)

# Load dataset
folder = Path(__file__).resolve().parent
dataset_path = folder / "data.csv"

if not dataset_path.exists():
    print("ERROR: data.csv was not found.")
    raise SystemExit

data = pd.read_csv(
    dataset_path,
    encoding_errors="ignore"
)
```

```
print("Dataset loaded successfully")
print("Original dataset size:", data.shape)

# Remove unnecessary columns
data = data.dropna(axis=1, how="all")

unnamed_columns = [
    column for column in data.columns
    if column.lower().startswith("unnamed")
]

data = data.drop(
    columns=unnamed_columns,
    errors="ignore"
)

data = data.drop(
    columns=["id"],
    errors="ignore"
)

# Convert diagnosis into numerical classes
data["diagnosis"] = (
    data["diagnosis"]
    .astype(str)
    .str.strip()
    .str.upper()
    .map({
        "B": 0,
```

```
        "M": 1
    })
)

data = data.dropna()
data["diagnosis"] = data["diagnosis"].astype(int)

feature_columns = [
    column
    for column in data.columns
    if column != "diagnosis"
]

for column in feature_columns:
    data[column] = pd.to_numeric(
        data[column],
        errors="coerce"
    )

data = data.dropna()
data = data.drop_duplicates()

print("Dataset size after cleaning:", data.shape)

# Calculate class probabilities
class_counts = data["diagnosis"].value_counts()
class_probabilities = data["diagnosis"].value_counts(
    normalize=True
)
```

```
print("\n---------- CLASS PROBABILITIES ----------")

print("Benign records:", class_counts.get(0, 0))
print("Malignant records:", class_counts.get(1, 0))

print(
    "P(Benign):",
    round(class_probabilities.get(0, 0) * 100, 2),
    "%"
)

print(
    "P(Malignant):",
    round(class_probabilities.get(1, 0) * 100, 2),
    "%"
)

# Input and target
X = data[feature_columns]
y = data["diagnosis"]

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42,
    stratify=y
```

```
)

# Train Gaussian Naive Bayes
model = GaussianNB()
model.fit(X_train, y_train)

# Evaluate model
test_predictions = model.predict(X_test)

accuracy = accuracy_score(
    y_test,
    test_predictions
)

print(
    "\nModel Accuracy:",
    round(accuracy * 100, 2),
    "%"
)

# Select one unseen test instance
new_instance = X_test.iloc[[0]]
actual_class = y_test.iloc[0]

prediction = model.predict(new_instance)[0]
posterior = model.predict_proba(new_instance)[0]

classes = list(model.classes_)
```

```
benign_probability = posterior[classes.index(0)]
malignant_probability = posterior[classes.index(1)]

print("\n---------- NEW INSTANCE PREDICTION ----------")

print(
    "Posterior P(Benign | Features):",
    round(benign_probability * 100, 2),
    "%"
)

print(
    "Posterior P(Malignant | Features):",
    round(malignant_probability * 100, 2),
    "%"
)

if prediction == 0:
    print("Predicted Class: BENIGN")
else:
    print("Predicted Class: MALIGNANT")

if actual_class == 0:
    print("Actual Class: BENIGN")
else:
    print("Actual Class: MALIGNANT")
```

Output Screenshot

```
Probability Theory Using Breast Cancer Dataset
------------------------------------------------------
Dataset loaded successfully
Original dataset size: (569, 33)
Dataset size after cleaning: (569, 31)

---------- CLASS PROBABILITIES ----------
Benign records: 357
Malignant records: 212
P(Benign): 62.74 %
P(Malignant): 37.26 %

Model Accuracy: 93.86 %

---------- NEW INSTANCE PREDICTION ----------
Posterior P(Benign | Features): 100.0 %
Posterior P(Malignant | Features): 0.0 %
Predicted Class: BENIGN
Actual Class: BENIGN
```

Result

The prior probabilities of benign and malignant classes were calculated successfully. Gaussian Naive Bayes used the class probabilities and patient-feature likelihoods to calculate posterior probabilities. The class with the higher posterior probability was selected as the diagnosis for the unseen patient record.

EXPERIMENT 2

Bayes Decision Theory Using SMS Spam Dataset

Problem Description

The objective is to apply Bayes' theorem to classify an SMS message as spam or ham. The model calculates the prior probability of each class and the conditional probability of words appearing in spam and ham messages.

Bayes' theorem is:

$$P(C \mid X) = \frac{P(X \mid C)P(C)}{P(X)}$$

Here, Crepresents either spam or ham, while Xrepresents the words in the message.

Dataset Description

The SMS Spam Collection contains **5,574 SMS messages** labelled as spam or ham. It combines legitimate messages and spam messages collected from several public research sources.

Dataset source: SMS Spam Collection – Kaggle

After extraction, keep either spam.csv or SMSSpamCollection in the program folder.

EXP-2

├── sms_spam_bayes.py

└── spam.csv

Algorithm

1. Load the labelled SMS Spam Collection dataset.
2. Remove missing and duplicate messages.
3. Calculate the prior probability of spam and ham.
4. Split the messages into training and testing sets.
5. Convert message words into numerical count features.
6. Train a Multinomial Naive Bayes classifier.
7. Calculate posterior probabilities and classify a new message.

Python Code

```
from pathlib import Path
```

```
import pandas as pd
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.metrics import accuracy_score, confusion_matrix
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import MultinomialNB


print("SMS Spam Classification Using Bayes Theorem")
print("-" * 55)

folder = Path(__file__).resolve().parent

csv_files = list(folder.glob("*.csv"))
text_file = folder / "SMSSpamCollection"

# Load CSV or text dataset
if csv_files:
    dataset_path = csv_files[0]

    data = pd.read_csv(
        dataset_path,
        encoding="latin-1",
        encoding_errors="ignore",
        on_bad_lines="skip"
    )

    print("Dataset found:", dataset_path.name)

    data = data.drop(
```

```
    columns=[
        column
        for column in data.columns
        if column.lower().startswith("unnamed")
    ],
    errors="ignore"
)

# Detect common column names
if "v1" in data.columns and "v2" in data.columns:
    data = data.rename(
        columns={
            "v1": "label",
            "v2": "message"
        }
    )

elif "Label" in data.columns and "Text" in data.columns:
    data = data.rename(
        columns={
            "Label": "label",
            "Text": "message"
        }
    )

elif "label" in data.columns and "text" in data.columns:
    data = data.rename(
        columns={
            "text": "message"
```

```
            }
        )

    elif "Category" in data.columns and "Message" in data.columns:
        data = data.rename(
            columns={
                "Category": "label",
                "Message": "message"
            }
        )

    else:
        print("ERROR: Label and message columns were not found.")
        print("Available columns:", data.columns.tolist())
        raise SystemExit

elif text_file.exists():
    data = pd.read_csv(
        text_file,
        sep="\t",
        names=["label", "message"],
        encoding_errors="ignore"
    )

    print("Dataset found:", text_file.name)

else:
    print("ERROR: SMS dataset was not found.")
    raise SystemExit
```

```
# Clean dataset
data = data[["label", "message"]].copy()
data = data.dropna()
data = data.drop_duplicates(subset=["message"])

data["label"] = (
    data["label"]
    .astype(str)
    .str.strip()
    .str.lower()
)

data["message"] = data["message"].astype(str)

data = data[
    data["label"].isin(["ham", "spam"])
]

print("Dataset loaded successfully")
print("Dataset size:", data.shape)

# Calculate prior probabilities
prior_probabilities = data["label"].value_counts(
    normalize=True
)

print("\n---------- PRIOR PROBABILITIES ----------")
```

```
print(
    "P(Ham):",
    round(prior_probabilities.get("ham", 0) * 100, 2),
    "%"
)

print(
    "P(Spam):",
    round(prior_probabilities.get("spam", 0) * 100, 2),
    "%"
)

# Input and target
X = data["message"]
y = data["label"]

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42,
    stratify=y
)

# Convert words into numerical counts
vectorizer = CountVectorizer(
    stop_words="english"
)
```

```
X_train_vector = vectorizer.fit_transform(X_train)
X_test_vector = vectorizer.transform(X_test)

# Train Multinomial Naive Bayes
model = MultinomialNB()
model.fit(X_train_vector, y_train)

predictions = model.predict(X_test_vector)

print(
    "\nModel Accuracy:",
    round(
        accuracy_score(y_test, predictions) * 100,
        2
    ),
    "%"
)

print("\nConfusion Matrix:")
print(confusion_matrix(y_test, predictions))

# New message prediction
new_message = input(
    "\nEnter an SMS message: "
)

new_message_vector = vectorizer.transform(
    [new_message]
```

```
)

prediction = model.predict(
    new_message_vector
)[0]

posterior = model.predict_proba(
    new_message_vector
)[0]

classes = list(model.classes_)

ham_probability = posterior[
    classes.index("ham")
]

spam_probability = posterior[
    classes.index("spam")
]

print("\n---------- POSTERIOR PROBABILITIES ----------")

print(
    "P(Ham | Message):",
    round(ham_probability * 100, 2),
    "%"
)

print(
```

```
    "P(Spam | Message):",
    round(spam_probability * 100, 2),
    "%"
)


if prediction == "spam":
    print("Final Classification: SPAM")
else:
    print("Final Classification: HAM")
```

Sample Input

Congratulations! You have won a free cash prize.

Call now to claim your reward.

Output Screenshot

```
SMS Spam Classification Using Bayes Theorem
--------------------------------------------------
Dataset found: spam.csv
Dataset loaded successfully
Dataset size: (5169, 2)

---------- PRIOR PROBABILITIES ----------
P(Ham): 87.37 %
P(Spam): 12.63 %

Model Accuracy: 98.45 %

Confusion Matrix:
[[898   5]
 [ 11 120]]

Enter an SMS message: hi today you won 1,00,000 rupees

---------- POSTERIOR PROBABILITIES ----------
P(Ham | Message): 1.49 %
P(Spam | Message): 98.51 %
Final Classification: SPAM
```

Result

Bayes' theorem was applied successfully to calculate the posterior probabilities of spam and ham. Multinomial Naive Bayes classified the entered message according to the class with the greater posterior probability.

EXPERIMENT 3

Information Theory Using Play Tennis Dataset

Problem Description

The objective is to calculate the entropy of the Play Tennis target and the Information Gain of every input attribute.

Entropy represents the uncertainty in the target:

$$H(S) = -\sum P(c)\log_2 P(c)$$

Information Gain measures how much an attribute reduces uncertainty:

$$IG(S, A) = H(S) - H(S \mid A)$$

The attribute with the highest Information Gain is considered the best attribute for splitting the dataset.

Dataset Description

The classic Play Tennis dataset contains categorical weather attributes such as outlook, temperature, humidity and wind. The target represents whether tennis should be played. It is commonly used to demonstrate entropy, Information Gain and Decision Tree learning.

Dataset source: Play Tennis Dataset – Kaggle

The downloaded file is:

PlayTennis.csv

Keep the files together:

EXP-3

├── information_gain_tennis.py

└── PlayTennis.csv

Algorithm

1. Load the Play Tennis classification dataset.
2. Remove identifier and empty columns.
3. Identify the target and input attributes.
4. Calculate the original entropy of the target.
5. Calculate conditional entropy for each attribute.
6. Calculate and rank the Information Gain values.

7. Display the attribute with the highest Information Gain.

Python Code

```
from pathlib import Path

import numpy as np
import pandas as pd



print("Information Gain Using Play Tennis Dataset")
print("-" * 55)

# Load dataset
folder = Path(__file__).resolve().parent

csv_files = list(folder.glob("*.csv"))

if not csv_files:
    print("ERROR: Play Tennis CSV file was not found.")
    raise SystemExit

dataset_path = csv_files[0]

data = pd.read_csv(
    dataset_path,
    encoding_errors="ignore",
    on_bad_lines="skip"
)

print("Dataset found:", dataset_path.name)
```

```
print("Original dataset size:", data.shape)
print("Available columns:", data.columns.tolist())

# Remove empty and unnamed columns
data = data.dropna(axis=1, how="all")

data = data.drop(
    columns=[
        column
        for column in data.columns
        if column.lower().startswith("unnamed")
    ],
    errors="ignore"
)

# Remove identifier columns
identifier_columns = [
    column
    for column in data.columns
    if column.strip().lower() in [
        "day",
        "id",
        "sl.no",
        "s.no"
    ]
]

data = data.drop(
    columns=identifier_columns,
```

```
    errors="ignore"
)

data = data.dropna()
data = data.drop_duplicates()

# Detect target column
normalized_columns = {
    column.lower()
    .replace(" ", "")
    .replace("_", ""): column

    for column in data.columns
}

possible_targets = [
    "playtennis",
    "play",
    "decision"
]

target_column = None

for name in possible_targets:
    if name in normalized_columns:
        target_column = normalized_columns[name]
        break

if target_column is None:
```

```
    target_column = data.columns[-1]

feature_columns = [
    column
    for column in data.columns
    if column != target_column
]

print("Target column:", target_column)
print("Input attributes:", feature_columns)

# Entropy function
def calculate_entropy(target):
    probabilities = target.value_counts(
        normalize=True
    )

    entropy = 0

    for probability in probabilities:
        if probability > 0:
            entropy -= (
                probability
                * np.log2(probability)
            )

    return entropy
```

```
# Information Gain function
def calculate_information_gain(
    dataset,
    feature,
    target
):
    original_entropy = calculate_entropy(
        dataset[target]
    )

    conditional_entropy = 0

    for value, group in dataset.groupby(feature):
        group_probability = len(group) / len(dataset)

        conditional_entropy += (
            group_probability
            * calculate_entropy(group[target])
        )

    information_gain = (
        original_entropy
        - conditional_entropy
    )

    return conditional_entropy, information_gain


# Original target entropy
```

```
target_entropy = calculate_entropy(
    data[target_column]
)

print(
    "\nTarget Entropy:",
    round(target_entropy, 6),
    "bits"
)

# Calculate gain for all attributes
results = []

for feature in feature_columns:
    conditional_entropy, gain = (
        calculate_information_gain(
            data,
            feature,
            target_column
        )
    )

    results.append({
        "Attribute": feature,
        "Conditional Entropy": conditional_entropy,
        "Information Gain": gain
    })

result_table = pd.DataFrame(results)
```

```
result_table = result_table.sort_values(
    by="Information Gain",
    ascending=False
).reset_index(drop=True)

result_table["Conditional Entropy"] = (
    result_table["Conditional Entropy"]
    .round(6)
)

result_table["Information Gain"] = (
    result_table["Information Gain"]
    .round(6)
)

print("\n---------- INFORMATION GAIN TABLE ----------")

print(
    result_table.to_string(index=False)
)

# Best attribute
best_attribute = result_table.iloc[0]

print("\n---------- BEST ATTRIBUTE ----------")

print(
    "Attribute with highest Information Gain:",
```

```
    best_attribute["Attribute"]
)

print(
    "Highest Information Gain:",
    best_attribute["Information Gain"],
    "bits"
)

# Save result
output_path = folder / "play_tennis_information_gain.csv"

result_table.to_csv(
    output_path,
    index=False
)

print("\nResults saved as:")
print(output_path.name)
```

Output Screenshot

```
Information Gain Using Play Tennis Dataset
------------------------------------------------------------
Dataset found: PlayTennis.csv
Original dataset size: (14, 5)
Available columns: ['outlook', 'temp', 'humidity', 'windy', 'play']
Target column: play
Input attributes: ['outlook', 'temp', 'humidity', 'windy']

Target Entropy: 0.940286 bits

---------- INFORMATION GAIN TABLE ----------
Attribute  Conditional Entropy  Information Gain
  outlook             0.693536         0.246750
 humidity             0.788450         0.151836
    windy             0.892159         0.048127
     temp             0.911063         0.029223

---------- BEST ATTRIBUTE ----------
Attribute with highest Information Gain: outlook
Highest Information Gain: 0.24675 bits

Results saved as:
play_tennis_information_gain.csv
```

Result

The entropy of the Play Tennis target and the Information Gain of every input attribute were calculated successfully. The attributes were ranked according to their Information Gain, and the attribute with the highest value was identified as the best feature for the first Decision Tree split.
